The Event rescorer example in Figure 1 depicts the verb-shuffled rescorer, where the model should learn that the progression *grow* → *began* → *fade* is a suitable verb ordering, but that *ended* → *lived* → *born* is unlikely. A simplified example for intra-shuffled from the same sample story would be the two phrases (within one sentence) `<A1> ent 12 <V> shifted` → `<A1> ent 12 <V> went <A2> to sleep`, as opposed to the reverse ordering. For inter-shuffled it would be the two full sentences `<A1> ent 11 <V> grew <A2> louder` → `<A1> ent 11 <V> fell <A2> silent`, again as opposed to the reverse.

Each of these has specific strengths and weaknesses. *Inter-sentence shuffling* is closest to work on Narrative Event Chains (Chambers and Jurafsky, 2008) and script-learning, which represent the fact that certain events are more likely to causally follow other events rather than precede them. However, since inter-sentence noising scopes globally over the entire plot structure, it is a harder task and may be difficult for the model to discriminate patterns. *Intra-sentence shuffling* is the same task but restricted to a more local scale, which makes the patterns clearer and more learnable but cannot capture long-distance Event Chains inter-sententially. It is also more sensitive to the style of a given story, as stories have a variable number of events per sentence. Finally, *verb-shuffling* focuses on verbs as the salient element of an event, and should teach both principles of verb ordering and of verb suitability for context, and avoid artifacts from reordering arguments. However, since verbs are shuffled naively, the task can in some cases be too easy due to differences in verb selectional preferences.[5]

Character Rescorers. We represent character trajectory by distinguishing which character should appear at what point in the story. We create training examples by taking each entity and all the preceding plot tokens up until the entity, and having the rescoring model choose between the true entity and a randomly sampled entity.[6] The character rescorer must then distinguish between points in the plot when a pre-existing entity (a character already present in the story) should reappear, and if so which one, or whether a new entity should be introduced. The intuition is that this should encode typical patterns of a character's actions and relationships in particular contexts.

The Figure 1 example encapsulates simplified correct character relationships (that a character should be asking something of another rather than of themself). This extends to much more complex character selection examples, as in the context: `<A0> ent 0 <V> saw <A1> the light of a campfire </s> <A1> ent 2 <V> laying <A2> there </s> # <A1> light <V> bouncing </s> <A0> ent ?` where the positive is `0` and the negative is `3`. Here the difference between the positive and negative captures whether or not it is natural to introduce a new character at this juncture (it is not, as in the gold plot a new character does not appear till a few sentences later).

Relevance Rescorers. We take an approach inspired by prior work on learning to discriminate between random and true continuations of story sentences (Holtzman et al., 2018). We consider pairs of prompts and plots, where a positive example is the true plot and the negative is a randomly selected plot from elsewhere in the training data. This prompt and plot pairing is a much more difficult task than pairing context and continuation sentences, since they are less closely connected. Once trained, this rescorer is expected to tell which kinds of plot words, verbs, and SRL patterns belong with which kinds of prompts. The example in Figure 1 shows a relevant and irrelevant sentence for the given prompt. During training, the rescorer will receive the entire gold story and an entire random story to match to a prompt (see Appendix B).

3.2 Rescoring Model Architecture

There is an inherent tension in training a useful rescoring model: discrimination tasks for which even simple models can perform well may have inherent data artifacts and therefore not be helpful for modeling plots. However, discrimination tasks that are so hard that all models have low accuracy are also likely to be unhelpful. We experiment with three different architectures for our rescorers. We start with ngram-baseline models[7] to better judge the baseline difficulty of a given task and take artifacts of data creation into account. This

[5] For instance, an exclusively transitive verb could be swapped into the space of an intransitive one, and the edit would be trivially recognisable based on grammatical patterns.

[6] In other words, each training example is a section of plot sequence $z_0 ... z_t$ where t is a randomly chosen entity, and the rescorer is given $z_0 ... z_{t-1}$ as context and must distinguish between the true and false z_t entities that follow.) So the length of context varies for each training example.

[7] Implemented in Scikit-learn (Pedregosa et al., 2011)

Plot Elements		XGB	CNN	RoBERTa
Relevance		55.2	56.0	**89.3**
Entity		64.1	76.0	**92.3**
Event	Inter-Shuffled	55.4	53.0	**96.6**
	Intra-Shuffled	58.5	75.0	**97.3**
	Verb-Shuffled	76.1	75.0	**95.8**

Table 2: Accuracy for training Rescorers using three different architectures.

is more informative than random chance accuracy. We also experiment with augmented versions of the CNN architectures used in Holtzman et al. (2018), and RoBERTa models (Liu et al., 2019), and find RoBERTa to have the best performance for each Aristotelian concept.

1. **XGBoost with ngrams:** We used n-grams in range of (1,4) as features and trained a XGBoost model with 200 gradient boosted trees with a maximum tree depth of 5.
2. **CNN with maxpooling:** We used a CNN-based architecture (Holtzman et al., 2018) but augmented it with BART position and subword encodings because our event tasks are ordered, so pooled or averaged representations that don't represent permutations differently would be indistinguishable.
3. **RoBERTa-large** (Liu et al., 2019) has shown excellent performance on various sentence pair classification tasks. We expect this large pre-trained language model to be more efficient in being able to discriminate between a well-formed sequence of words and a poorer one. To this end we finetune RoBERTa-large with a classification specific final layer as the final option to build rescorer models.

Accuracies for different rescorer architectures by aspect are shown in Table 2. As we hypothesised from the nature of many of the tasks (Section 3.1), the inter-sentence shuffled task is more difficult because the noising is global. This is reflected in the barely above chance scores of the ngram-baseline. The Verb-shuffling high ngram-baseline performance shows that our suspicions about this task being easier were also correct. Intra-shuffling was the only surprise, and turned out to be more difficult than we expected and to have the largest gap between baseline ngram and CNN performance. RoBERTa scores are high across the board, so we use RoBERTa for all models in the final system.

3.3 Mixture Weight Training and Ablations

We learn optimal weights for rescorers online during decoding using a held-out validation set V, and use these weights during inference via sampling. We minimize Margin Ranking Loss of the negative log probability of each validation sample between the gold ($\mathbf{z}$) and hypothesised ($\hat{\mathbf{z}}$) plot structures.

$$\mathcal{L}_{\lambda}^{mix} = \sum_{\mathbf{x},\mathbf{z}\in V} \sum_{i=1}^{n} \max(0, -(f_\lambda(\mathbf{x}, \mathbf{z}_{<i}) - f_\lambda(\mathbf{x}, \hat{\mathbf{z}}_{<i})) \tag{3}$$

where i indexes the word position, n denotes the plot length, f_λ is the same as in Equation 1, and we are training the λ weights with this objective.[8] We train mixture weights both for combinations of rescorers and for ablations using each rescorer individually, to isolate the contribution of each one.

Mixture weight training accuracy is in Table 3,[9] which we report as Ranking Accuracy, the number of samples where the generation has higher probability than the gold. There we also include our automated plot metrics on the validation set for each ablation (further detail on those metrics in Section 4.2). As Table 3 shows, Inter-event is the strongest of the individual rescorers, though all five together achieve the best performance. This seems to indicate that each method of creating negative event examples is encoding a separate helpful piece of information, rather than one of them alone being the best approach.

Given that the ensemble of many rescorers was best, this suggests further investigation into the interactions between rescorers, and into whether there is a pareto front and in which cases certain rescorers dominate. Due to the difficulty of human evaluation of long stories and the accompanying funding constraints, we leave this as a suggestion for interesting future work to analyse. We thus select the ensemble of all 5 rescorers with optimal learned mixture weights for our final Aristotelian Plot System.

4 Experimental Setup

Dataset. We use the Writing Prompts dataset (Fan et al., 2018), which is a large collection of user-generated stories along with their associated prompts from Reddit, to benchmark our models. It

[8] We experiment with learning on each token (in which case, each validation sample will contain a few hundred updates) vs. on each complete sample (an entire plot). We find both approaches to perform similarly (with every-token gaining 1% accuracy) and use an every-token approach in for our final system. Mixture weight hyperparameters can be found in Appendix A.4

[9] Our 4-scorer ensemble subtracts out the Intra-event model as it is conceptually a more local version of the Inter-event model, and has a lesser effect on Ranking Accuracy

Rescorer	RA ↑	V:T ↑	E ↑
All 5	**0.43**	**2.07**	7.15
All 4 (-intra)	0.35	1.72	8.78
Inter-event	0.37	1.76	8.87
Intra-event	0.16	2.01	6.58
Verb-event	0.15	1.66	8.89
Entity	0.17	1.68	**9.18**
Relevance	0.17	1.71	8.89

Table 3: Ranking accuracy (RA) and generated plot metrics i.e Verb:Token ratio (V:T), Entities(E) for rescorer ablations

is particularly suited to this task since it is both hierarchical (contains pairs of titles and stories, which enables the use of a plot as an interim step) and contains many diverse long-form stories that are very challenging to learn to structure.[10]

4.1 Baselines

We compare our generations to the two strongest recent story generation systems as well as two ablated versions of our own system.

Targeted Common Sense Grounding Model. Mao et al. (2019) propose a multi-task learning scheme to achieve quantitatively better common sense reasoning in pre-trained language models by leveraging auxiliary training signals from datasets designed to provide common sense grounding.

Knowledge-Enhanced Commonsense Model. Guan et al. (2020) devise a knowledge-enhanced pretraining model for commonsense story generation leveraging external knowledge bases on the ROCStories dataset (Mostafazadeh et al., 2016). To further capture causal and temporal dependencies between sentences in a reasonable story, they employ multi-task learning which combines a discriminative objective to distinguish true and fake stories during fine-tuning.

Prompt to Story. This fine tunes the BART model directly with the prompt and story pairs without access to a plot structure.

Naive Plot. This utilizes a plot structure to write a story, but does not incorporate the Aristotelian Rescorer ensemble.

We do not compare to the strong system of Fan et al. (2019), as they do not release their code and the authors were unable to make their output available after being contacted.[11] However Guan et al. (2020) is one of our baselines and achieves better results on Mostafazadeh et al. (2016) than their approach. Our **Naive Plot** baseline is most comparable and quite similar to their system, save that it lacks their verb attention, but leverages powerful BART pretraining.

4.2 Metrics

We use a combination of human and automatic metrics to evaluate all systems. Automatic metrics are evaluated on 1000 randomly selected prompts and their associated plot structures and stories from the WritingPrompts test set. As automatic metrics are known to be overly coarse for open domain NLG (Novikova et al., 2017) we also report human evaluation results on 110 stories, with each story evaluated by 3 separate judges.[12] All metrics are averaged over all stories within the evaluation set.

Plot Structure Metrics. For plot structures we use purely automated metrics, as it is difficult for humans to read the abstract interim representation of SRL tags and entity anonyimisation:

- Vocab:Token Ratio
- Entities per plot

We choose to focus on these metrics as indicative of performance as they show the largest gap between gold and generated stories. *Vocab:Token Ratio* is commonly used as a measure of originality and diversity of content for stories, and can be readily applied to plots. *Entities per plot* we found to be a good metric as a reasonableness check, and for ablations, as weak models tend to have too few entities. Both metrics also relate to repetition, a known weakness of neural generated stories in comparison to human stories even with large pretrained transformers.

Automatic Story Metrics.

- Vocab:Token Ratio
- Unique Verbs & % Diverse Verbs: % of verbs not in the top 5 most frequent[13].
- Intra-story Trigram Repetition: trigram repetition *within* a story
- Inter-story Trigram Repetition: trigram repetition *between* stories

We again use Vocab:Token ratio as a rough diversity metric. We report also inter-story trigram repetition

[10] Dataset statistics are in Appendix A.2

[11] Code for Fan et al. (2018), which does *not* use plots, is available but is trivially beaten by any recent story generation system.

[12] Workers were paid $12/hr. They were given content instructions and told to disregard punctuation and spelling. Further details on compensation and instructions in Appendix E.

[13] We report these for comparability to Fan et al. (2019); as they do we identify verbs via `https://spacy.io/`